

Generador de Frecuencias utilizando el microcontrolador ATmega328P

Karla Yamilet Martínez Pichal

Universidad Nacional Autónoma de México, Facultad de Estudios Superiores Cuautitlán
Microcontroladores

28 de septiembre de 2026

Resumen

En el presente documento describimos el desarrollo de un generador de señales cuadradas operando a frecuencias de 100 kHz, 500 kHz, 1 MHz y 2 MHz. Se utilizó el microcontrolador ATmega328P con un oscilador de cristal de 16 MHz. La selección de la frecuencia se realiza mediante el sondeo (*polling*) de dos pines de entrada (PB3 y PB4), obteniendo la señal de salida en el pin PB5. Se detalla la implementación de una arquitectura de software híbrida en lenguaje ensamblador, utilizando subrutinas clásicas para bajas frecuencias y técnicas de optimización algorítmica documentadas en la literatura de arquitectura de computadoras [2], como el desenrollado de bucles (*loop unrolling*). Asimismo, se implementó la evaluación dinámica del registro de estado (SREG) fundamentada en los tiempos de ejecución del conjunto de instrucciones AVR [3], logrando así superar las limitaciones físicas del microcontrolador a altas frecuencias.

1. Introducción

Los sistemas basados en microcontroladores de la familia AVR ejecutan la mayor parte de sus instrucciones en un solo ciclo de reloj. Sin embargo, al sintetizar señales de alta frecuencia mediante software, el tiempo invertido en leer los puertos de entrada, evaluar condiciones lógicas y ejecutar saltos puede exceder el periodo de la onda deseada.

Este proyecto requiere conmutar una señal a 2 MHz utilizando un reloj principal de 16 MHz. Esto impone un presupuesto estricto de ciclos de máquina para la ejecución de rutinas en ensamblador. Aunque los temporizadores por hardware (Timers) reducen la carga computacional de la CPU, las especificaciones de diseño exigen que la salida sea por el pin PB5, el cual no está mapeado a las salidas de comparación directa de los Timers internos del microcontrolador ATmega328P. Por lo tanto, fue necesario implementar la conmutación por software

de alta precisión.

2. Diseño del Sistema

El sistema emplea tres pines del puerto B. PB5 se configuró como salida digital, mientras que PB3 y PB4 se configuraron como entradas habilitando sus resistencias *pull-up* internas.

2.1. Arquitectura Híbrida de Software

Para satisfacer los requerimientos temporales, el programa se dividió en dos paradigmas de control estructural:

- **Frecuencia Baja (100 kHz):** Se dispone de 160 ciclos por periodo. Se empleó un diseño modular mediante las instrucciones `rcall` y `ret`, facilitando la reutilización de código mediante subrutinas de retardo dinámicas.
- **Frecuencias Altas (500 kHz y 1 MHz):** Las instrucciones de salto a subrutina consumen demasiados ciclos. Se reemplazaron por la instrucción de conmutación directa `sbi PINB, 5`, intercalada con pequeños bucles de evaluación en línea.

3. Análisis de Resultados

3.1. El límite físico de los 2 MHz

La generación de una frecuencia de 2 MHz requiere 4 millones de conmutaciones por segundo. Operando a 16 MHz, el límite físico se calcula de la siguiente manera:

$$\text{Ciclos} = \frac{16,000,000 \text{ Hz}}{4,000,000 \text{ conmutaciones/s}} = 4 \text{ ciclos} \quad (1)$$

El procesador dispone de exactamente 4 ciclos de reloj para cambiar el estado del pin. La instrucción `sbi`

PINB, 5 consume 2 ciclos, y un salto incondicional `rjmp` consume 2 ciclos. Intentar insertar una instrucción de lectura (`in` PINB, 1 ciclo) excedería el presupuesto temporal, reduciendo la frecuencia y fallando los parámetros del sistema.

Algorithm 1 Lógica general de selección de frecuencia

```

1: Inicializar Puntero de Pila (Stack Pointer)
2: DDRB ← Configurar PB5 como salida
3: PORTB ← Habilitar pull-up en PB3, PB4
4: loop
5:   temp ← Leer PINB
6:   Aplicar máscara 0b0001_1000 a temp
7:   if temp == 00 then
8:     Ejecutar rutina 100 kHz (breq)
9:   end if
10:  if temp == 01 then
11:    Ejecutar rutina 500 kHz (breq)
12:  end if
13:  if temp == 10 then
14:    Ejecutar rutina 1 MHz (breq)
15:  end if
16:  if temp == 11 then
17:    Ejecutar rutina 2 MHz (breq)
18:  end if
19: end loop

```

3.2. Justificación Teórica de la Optimización (Loop Unrolling y SREG)

Para lograr la estabilidad temporal del sistema a 2 MHz sin degradar la frecuencia de muestreo de los puertos, se aplicó la técnica de *desenrollado de bucles* (Loop Unrolling). Según establecen Patterson y Hennessy en su obra canónica de arquitectura de computadoras, el desenrollado de bucles es una técnica de optimización donde “el cuerpo del bucle se replica múltiples veces [...] para reducir la sobrecarga de las instrucciones de control de flujo y exponer el paralelismo a nivel de instrucción” [2]. Al replicar la instrucción de conmutación (`sbi`) cuatro veces, se eliminó la penalización de ciclos que impondría un bucle cerrado tradicional.

Adicionalmente, el cuadro exacto de los ciclos de máquina (*cycle-counting*) se fundamentó en la evaluación asimétrica del registro de estado (SREG). De acuerdo con el *AVR[®] Instruction Set Manual* de Microchip [3], las instrucciones de bifurcación condicional como `brne` (Branch if Not Equal) y `breq` (Branch if Equal) presentan tiempos de ejecución variables: “1 ciclo si la condición es falsa, y 2 ciclos si la condición es verdadera” (Secciones de instrucción BRNE y BREQ, págs. 47 y 45 respectivamente).

Al forzar el flujo del programa a través de condiciones intencionalmente falsas durante la operación normal a 2

MHz, se consumió exactamente un ciclo de reloj por instrucción. Los retardos estructurales restantes se completaron con las operaciones aritméticas `clc` (Clear Carry) y `sec` (Set Carry), las cuales el fabricante especifica textualmente con un tiempo de ejecución fijo de “1 ciclo” [3], permitiendo el ajuste temporal perfecto en la Unidad Lógico Aritmética (ALU) sin alterar la bandera de cero (Z) necesaria para la evaluación de los puertos.

Para evadir la latencia de las instrucciones de control y mantener el muestreo continuo de los puertos, se reestructuró la rutina mediante la técnica de *Desenrollado de Bucle* (Loop Unrolling). Esta estrategia permitió linealizar el flujo de ejecución en una ventana continua de 16 ciclos de máquina, optimizando el rendimiento de la CPU sin recurrir a instrucciones de retardo pasivo.

Algorithm 2 Desenrollado de bucle para 2 MHz

1: <code>sbi</code> PINB, 5	▷ Conmutación 1 (2 ciclos)
2: <code>in</code> temp, PINB	▷ Lectura (1 ciclo)
3: <code>clc</code>	▷ Retardo estructural (1 ciclo)
4: <code>sbi</code> PINB, 5	▷ Conmutación 2 (2 ciclos)
5: <code>andi</code> temp, 0x18	▷ Enmascarado (1 ciclo)
6: <code>sec</code>	▷ Retardo estructural (1 ciclo)
7: <code>sbi</code> PINB, 5	▷ Conmutación 3 (2 ciclos)
8: <code>cpi</code> temp, 0x18	▷ Comparación (1 ciclo)
9: <code>clc</code>	▷ Retardo estructural (1 ciclo)
10: <code>sbi</code> PINB, 5	▷ Conmutación 4 (2 ciclos)
11: <code>breq</code> repite_bucle	▷ Evaluación final (2 ciclos)

Entre cada instrucción de conmutación, se insertaron operaciones aritméticas (`clc`, `sec`) que consumen un ciclo exacto en la Unidad Lógico Aritmética (ALU) sin modificar la bandera de cero (Z) del registro de estado (SREG). Esto garantiza la separación perfecta de 4 ciclos entre conmutaciones, asegurando una onda cuadrada simétrica a 2.0 MHz.

4. Conclusión

El desarrollo de esta primera actividad evidenció cómo el rendimiento de un sistema embebido está estrictamente limitado por la arquitectura física del microprocesador. Durante la fase de diseño y simulación, resultó revelador comprobar que la lógica de programación estructurada convencional colapsaba al intentar alcanzar la señal de 2 MHz, ya que el microcontrolador simplemente carecía del tiempo físico necesario para ejecutar saltos largos y leer los puertos simultáneamente.

La técnica de desenrollado de bucles va más allá de un concepto teórico. Lograr una onda cuadrada estable y fluida sin depender de los temporizadores internos demostró que, con un análisis matemático riguroso del lenguaje ensamblador, es posible emular la precisión del

hardware exclusivamente mediante la optimización del software.

Referencias

- [1] Microchip Technology Inc., *ATmega48A/PA/88A/PA/168A/PA/328/P Datasheet*, 2020. Disponible en: <https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061A.pdf>
- [2] D. A. Patterson y J. L. Hennessy, *Estructura y diseño de computadores: la interfaz hardware/software*, 4ta ed. Barcelona, España: Editorial Reverté, 2011.
- [3] Microchip Technology Inc., *AVR® Instruction Set Manual*, 2016. Disponible en: <https://ww1.microchip.com/downloads/en/devicedoc/atmel-0856-avr-instruction-set-manual.pdf>